

Ejercicios Prácticos - Unidad 1, Sesión 2

Large Language Models en Profundidad

Ejercicio 1: Comparativa de LLMs en Acción

Metadata

- **Duración estimada:** 40 minutos
- **Tipo:** Exploración
- **Modalidad:** Parejas
- **Dificultad:** Básica
- **Prerequisitos:** Cuentas gratuitas en ChatGPT, Claude, Gemini

Contexto

Cada LLM tiene fortalezas y debilidades. Este ejercicio desarrolla intuición práctica sobre sus diferencias.

Objetivo de Aprendizaje

- Comparar respuestas de diferentes LLMs al mismo prompt
- Identificar diferencias en estilo, precisión y formato
- Desarrollar criterio para seleccionar modelos según tarea

Instrucciones

Setup (5 min)

Asegúrate de tener acceso a al menos 3 de estos:

- [ChatGPT](#) (gratuito)
- [Claude](#) (gratuito)
- [Gemini](#) (gratuito)
- [Mistral Chat](#) (gratuito)
- [HuggingFace Chat](#) (gratuito, acceso a modelos open-source como Llama, Qwen, Mistral, etc.)

Parte A: Tarea de Conocimiento (10 min)

Prompt idéntico para todos:

Explica el teorema de Bayes de forma simple, incluyendo un ejemplo práctico de uso en machine learning.

Evalúa cada respuesta en (1-5):

Modelo	Claridad	Precisión	Ejemplo útil	Longitud apropiada
ChatGPT				
Claude				
Gemini				

Parte B: Tarea de Razonamiento (10 min)

Prompt idéntico para todos:

Un caracol está en el fondo de un pozo de 10 metros. Cada día sube 3 metros pero por la noche resbala 2 metros. ¿En cuántos días llega arriba? Explica tu razonamiento paso a paso.

Modelo	Respuesta correcta?	Razonamiento claro?	Nota
ChatGPT			
Claude			
Gemini			

Respuesta correcta: 8 días (día 8 sube 3m y llega a 10m antes de que anochezca)

Parte C: Tarea de Código (10 min)

Prompt idéntico para todos:

Escribe una función en Python que determine si un número es primo. Incluye tests y documentación.

Modelo	Código correcto?	Tests incluidos?	Documentación?	Eficiencia
ChatGPT				
Claude				
Gemini				

Parte D: Síntesis (5 min)

Completa esta matriz resumen:

Tarea	Mejor modelo	¿Por qué?
Explicaciones		

Tarea	Mejor modelo	¿Por qué?
Razonamiento		
Código		

Preguntas de Reflexión

1. ¿Algún modelo fue consistentemente mejor en todo?
2. ¿Notaste diferencias en el "tono" o estilo de las respuestas?
3. Si tuvieras que elegir un solo modelo para uso general, ¿cuál sería y por qué?

Entregable

- Tablas comparativas completadas
- Respuestas a preguntas de reflexión
- Capturas de pantalla de respuestas más interesantes

Ejercicio 2: Explorando la Tokenización

Metadata

- **Duración estimada:** 25 minutos
- **Tipo:** Exploración/Programación
- **Modalidad:** Individual
- **Dificultad:** Intermedia
- **Prerequisitos:** Python básico

Contexto

La tokenización afecta directamente el coste, el límite de contexto y algunas capacidades del modelo. Entenderla es esencial.

Objetivo de Aprendizaje

- Visualizar cómo se tokeniza texto real
- Comprender la relación tokens-palabras
- Identificar casos problemáticos de tokenización

Recursos

Usa el tokenizador oficial de OpenAI:

- [OpenAI Tokenizer](#)
- O la biblioteca `tiktoken`:

```
# Instalación
# pip install tiktoken
```

```
import tiktoken

# Tokenizador de GPT-4
enc = tiktoken.encoding_for_model("gpt-4")

def analizar_texto(texto):
    tokens = enc.encode(texto)
    print(f"Texto: '{texto}'")
    print(f"Número de tokens: {len(tokens)}")
    print(f"Tokens: {tokens}")
    print(f"Decodificado: {[enc.decode(t)] for t in tokens}")
    print(f"Ratio tokens/palabras: {len(tokens)/len(texto.split()):.2f}")
    print()

# Ejemplos
analizar_texto("Hello, world!")
analizar_texto("Hola, mundo!")
analizar_texto("El procesamiento de lenguaje natural es fascinante")
```

Instrucciones

Parte A: Análisis Básico (10 min)

Analiza los siguientes textos y completa la tabla:

Texto	Palabras	Tokens	Ratio
"The quick brown fox jumps"			
"El rápido zorro marrón salta"			
"Der schnelle braune Fuchs springt"			
"机器学习是人工智能的一个分支"			

Pregunta: ¿Qué idiomas son más "eficientes" en tokens?

Parte B: Casos Especiales (10 min)

Analiza estos casos y explica los resultados:

```
# Números
analizar_texto("123")
analizar_texto("123456789")
analizar_texto("12345678901234567890")

# Código
analizar_texto("def hello_world():")
analizar_texto("function helloWorld() {")

# Emojis
analizar_texto("I love 🎉 parties!")
```

```
analizar_texto("🎉🎁🎊🎈🍰")

# Palabras raras
analizar_texto("Supercalifragilisticexpialidocious")
analizar_texto("antidisestablishmentarianism")
```

Preguntas:

1. ¿Cómo afecta la longitud de números a la tokenización?
2. ¿Cuántos tokens usa cada emoji aproximadamente?
3. ¿Qué pasa con palabras muy largas?

Parte C: Cálculo de Costes (5 min)

Dado:

- GPT-4 cuesta \$0.03 por 1K tokens de input
- Tu texto tiene 5,000 palabras en español

Estima:

1. ¿Cuántos tokens aproximadamente?
2. ¿Cuál sería el coste de procesar ese texto?

Solución de Referencia

► Ver código completo

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4")

def analisis_completo(texto, descripcion=""):
    tokens = enc.encode(texto)
    palabras = len(texto.split())

    print(f"=== {descripcion} ===" if descripcion else "")
    print(f"Texto: '{texto[:50]}...' " if len(texto) > 50 else f"Texto: '{texto}'")
    print(f"Caracteres: {len(texto)}")
    print(f"Palabras: {palabras}")
    print(f"Tokens: {len(tokens)}")
    print(f"Ratio tokens/palabras: {len(tokens)/max(palabras,1):.2f}")
    print(f"Tokens desglosados: {[enc.decode([t]) for t in tokens[:10]]}...")
    print()

    return len(tokens)

# Idiomias
analisis_completo("The quick brown fox jumps over the lazy dog", "Inglés")
```

```
analisis_completo("El rápido zorro marrón salta sobre el perro perezoso",  
"Español")  
analisis_completo("Der schnelle braune Fuchs springt über den faulen  
Hund", "Alemán")  
  
# Números  
analisis_completo("123", "Número corto")  
analisis_completo("123456789012345", "Número largo")  
  
# Cálculo de costes  
texto_largo = "Esta es una oración de ejemplo. " * 100  
tokens = analisis_completo(texto_largo, "Texto largo")  
print(f"Coste estimado (GPT-4 input): ${tokens * 0.03 / 1000:.4f}")
```

Ejercicio 3: Experimentando con Parámetros de Generación

Metadata

- **Duración estimada:** 30 minutos
- **Tipo:** Exploración
- **Modalidad:** Individual
- **Dificultad:** Intermedia
- **Prerequisitos:** Acceso a API de OpenAI (o OpenRouter gratuito) o playground

Contexto

Los parámetros de generación (temperatura, top-p, top-k) tienen impacto significativo en la calidad y estilo de las respuestas.

Objetivo de Aprendizaje

- Experimentar con diferentes valores de temperatura
- Observar efecto de top-p en diversidad
- Desarrollar intuición para elegir parámetros según tarea

Recursos

Puedes usar:

- [OpenAI Playground](#) (requiere cuenta con crédito)
- [Google AI Studio](#) (gratuito)
- [HuggingFace Chat](#) (gratuito, algunos modelos permiten ajustar parámetros en la configuración del chat)
- Código con API (OpenAI u OpenRouter):

Con OpenAI:

```
from openai import OpenAI
```

```

client = OpenAI() # Requiere OPENAI_API_KEY

def generar(prompt, temperatura=1.0, top_p=1.0, n=3):
    """Genera n respuestas con los parámetros dados"""
    responses = []
    for _ in range(n):
        response = client.chat.completions.create(
            model="gpt-3.5-turbo",
            messages=[{"role": "user", "content": prompt}],
            temperature=temperatura,
            top_p=top_p,
            max_tokens=100
        )
        responses.append(response.choices[0].message.content)
    return responses

```

Con OpenRouter (gratis):

```

from openai import OpenAI

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key="tu-api-key-de-openrouter", # Regístrate gratis en
    openrouter.ai
)

def generar(prompt, temperatura=1.0, top_p=1.0, n=3):
    """Genera n respuestas con los parámetros dados (modelos gratuitos)"""
    responses = []
    for _ in range(n):
        response = client.chat.completions.create(
            model="google/gemma-3-4b-it:free", # u otro modelo gratuito
            messages=[{"role": "user", "content": prompt}],
            temperature=temperatura,
            top_p=top_p,
            max_tokens=100
        )
        responses.append(response.choices[0].message.content)
    return responses

```

Instrucciones

Parte A: Efecto de la Temperatura (15 min)

Usa el prompt: "Escribe el inicio de una historia de ciencia ficción en una oración."

Genera 3 respuestas con cada configuración:

Temperatura	Respuesta 1	Respuesta 2	Respuesta 3	Observación
-------------	-------------	-------------	-------------	-------------

Temperatura	Respuesta 1	Respuesta 2	Respuesta 3	Observación
0.0				
0.5				
1.0				
1.5				

Preguntas:

1. ¿Qué pasa con la diversidad al aumentar temperatura?
2. ¿A partir de qué temperatura empiezas a ver incoherencia?
3. ¿Qué temperatura usarías para escribir un email formal?

Parte B: Efecto de Top-p (10 min)

Con temperatura fija en 1.0, prueba diferentes top-p:

Prompt: "Lista 5 aplicaciones innovadoras de inteligencia artificial"

Top-p	Observación sobre diversidad
0.5	
0.7	
0.9	
1.0	

Parte C: Combinaciones Prácticas (5 min)

Basado en tus experimentos, sugiere configuraciones para:

Caso de uso	Temperatura	Top-p	Justificación
Generación de código			
Escritura creativa			
Responder FAQs			
Brainstorming			

Solución de Referencia

► Ver configuraciones recomendadas

Caso de uso	Temperatura	Top-p	Justificación
Generación de código	0.2	0.9	Precisión sobre creatividad
Escritura creativa	0.8-1.0	0.95	Diversidad y originalidad

Caso de uso	Temperatura	Top-p	Justificación
Responder FAQs	0.3	0.9	Consistencia y precisión
Brainstorming	1.0-1.2	0.95	Máxima diversidad de ideas

Ejercicio 4: Simulación del Ciclo de Vida de un LLM

Metadata

- **Duración estimada:** 35 minutos
- **Tipo:** Análisis/Diseño
- **Modalidad:** Grupal (3-4 personas)
- **Dificultad:** Avanzada
- **Prerequisitos:** Teoría de pre-entrenamiento, fine-tuning, RLHF

Contexto

Imagina que tu equipo debe planificar el desarrollo de un LLM especializado para soporte técnico de una empresa de software.

Objetivo de Aprendizaje

- Aplicar conocimientos teóricos a un caso práctico
- Tomar decisiones de diseño justificadas
- Considerar trade-offs de recursos y calidad

Enunciado

Escenario: TechCorp quiere desarrollar "TechBot", un asistente de IA para responder consultas técnicas sobre sus productos (3 productos de software, ~500 páginas de documentación total).

Recursos disponibles:

- Presupuesto: \$50,000
- Tiempo: 3 meses
- Equipo: 2 ML engineers, 1 data scientist
- Hardware: 4 GPUs A100

Instrucciones

Fase 1: Decisiones de Arquitectura (10 min)

Discutan y elijan:

1. **Base del modelo:**
 - ☐ Entrenar desde cero
 - ☐ Fine-tune de LLaMA (open-source)
 - ☐ Fine-tune vía API de OpenAI/Anthropic
 - ☐ Usar modelo vía API sin fine-tune (solo prompting)

Justificación:**2. Tamaño del modelo** (si aplica):

- ☐ 7B parámetros
- ☐ 13B parámetros
- ☐ 70B parámetros

Justificación:**Fase 2: Estrategia de Datos (10 min)**

Diseñen la estrategia de datos:

1. Fuentes de datos para fine-tuning:

- Documentación existente (500 páginas)
- Tickets de soporte históricos (si disponibles)
- FAQs
- Otros: _____

2. Formato de datos:

Ejemplo de par instrucción-respuesta:

Instrucción: _____

Respuesta: _____

3. Cantidad estimada de ejemplos necesarios: _____**4. ¿Incluirán RLHF?**

- ☐ Sí - ¿Cómo obtendrán feedback humano?
- ☐ No - ¿Por qué?

Fase 3: Plan de Implementación (10 min)

Creen un plan de alto nivel:

Semana	Actividad	Entregable
1-2		
3-4		
5-6		
7-8		
9-10		
11-12		

Fase 4: Evaluación (5 min)

Definan métricas de éxito:

Métrica	Objetivo	Cómo medirla
Precisión en respuestas		
Satisfacción de usuarios		
Tiempo de respuesta		
Tasa de escalación a humanos		

Presentación

Cada grupo presenta su plan (3 min) respondiendo:

- 1. ¿Qué enfoque eligieron y por qué?
- 2. ¿Cuál es el mayor riesgo de su plan?
- 3. ¿Qué harían diferente con el doble de presupuesto?

Solución de Referencia

► Ver una posible solución

Enfoque recomendado para presupuesto limitado:

- 1. **Base:** Fine-tune de LLaMA 7B vía QLoRA
 - Open-source, ejecutable en hardware disponible
 - Suficiente capacidad para dominio específico
- 2. **Datos:**
 - Convertir documentación a ~2,000 pares Q&A
 - Usar tickets históricos para ~1,000 ejemplos adicionales
 - Total: ~3,000 ejemplos de alta calidad
- 3. **Sin RLHF formal:**
 - Demasiado costoso para el presupuesto
 - En su lugar: evaluación iterativa con equipo de soporte
- 4. **Timeline:**
 - Semanas 1-4: Preparación de datos
 - Semanas 5-8: Fine-tuning y experimentos
 - Semanas 9-10: Evaluación y ajustes
 - Semanas 11-12: Despliegue piloto

Ejercicio 5: Quiz de Conceptos LLMs

Metadata

- **Duración estimada:** 15 minutos
- **Tipo:** Evaluación
- **Modalidad:** Individual
- **Dificultad:** Variada

Instrucciones

Responde sin consultar material. Verifica respuestas al final.

Pregunta 1 (Básica)

¿Cuál es la principal diferencia entre un modelo "open weights" y uno "fully open source"?

- a) No hay diferencia b) Open weights solo publica los pesos, no necesariamente el código de entrenamiento ni datos c) Open weights es más restrictivo legalmente d) Fully open source no permite uso comercial

Pregunta 2 (Intermedia)

Si un modelo tiene context window de 8,000 tokens y tu prompt tiene 7,000 tokens, ¿cuántos tokens puede tener la respuesta como máximo?

- a) 8,000 tokens b) 1,000 tokens c) 15,000 tokens d) Depende del modelo

Pregunta 3 (Intermedia)

¿Por qué el RLHF fue crucial para el éxito de ChatGPT?

- a) Hizo el modelo más rápido b) Alineó las respuestas con preferencias humanas de utilidad y seguridad c) Aumentó el número de parámetros d) Redujo los costes de entrenamiento

Pregunta 4 (Avanzada)

Un tokenizador BPE para inglés típicamente produce más tokens para:

- a) Texto en inglés técnico b) Texto en español cotidiano c) Código Python bien formateado d) Números de un solo dígito

Pregunta 5 (Avanzada)

Si configuras temperatura=0 y ejecutas el mismo prompt 10 veces, ¿obtendrás siempre la misma respuesta?

- a) Sí, siempre idéntica b) Casi siempre, pero puede haber variaciones por aritmética de punto flotante c) No, la temperatura solo afecta la velocidad d) Depende del modelo

Pregunta 6 (Básica)

¿Qué benchmark mide la capacidad de un LLM para resolver problemas de programación?

a) MMLU b) HumanEval c) HellaSwag d) TruthfulQA

Respuestas

► Ver respuestas

1. **b)** - Open weights publica pesos pero no necesariamente código/datos completos
2. **b)** - Context window = prompt + respuesta, máximo 8000-7000 = 1000 tokens
3. **b)** - RLHF alineó el modelo con preferencias humanas
4. **b)** - Tokenizadores optimizados para inglés son menos eficientes en otros idiomas
5. **b)** - Casi siempre igual, pero variaciones por computación paralela en GPU
6. **b)** - HumanEval mide generación de código con tests

Ejercicio Extra: Investigación de Arquitecturas

Metadata

- **Duración estimada:** 45 minutos (tarea para casa)
- **Tipo:** Investigación
- **Modalidad:** Individual
- **Dificultad:** Avanzada

Enunciado

Investiga **Mixture of Experts (MoE)**, la arquitectura usada por modelos como Mixtral, DeepSeek V3 y LLaMA 4.

Puntos a Cubrir

1. **Concepto básico:** ¿Qué es un "experto" y cómo se selecciona?
2. **Ventajas:** ¿Por qué es más eficiente que arquitecturas densas?
3. **Desafíos:** ¿Qué problemas de entrenamiento presenta?
4. **Comparativa:** ¿Cómo se compara un MoE de 8x7B vs un modelo denso de 70B?

Recursos Sugeridos

- Paper: "Switch Transformers" (Google, 2021)
- Paper: "Mixtral of Experts" (Mistral AI, 2024)
- Blog posts de Hugging Face sobre MoE

Entregable

Documento de 1-2 páginas con:

- Explicación del concepto
- Diagrama simple de la arquitectura
- Tabla comparativa con arquitectura densa
- Tu opinión: ¿Es MoE el futuro de los LLMs?